

Milestone 4: ExoHabitAI System Integration, Interface, and Deployment

1. Theory About the Code

The ExoHabitAI project is an advanced machine learning pipeline designed to predict exoplanet habitability based on planetary and stellar metrics. The core methodology bridges feature engineering, robust handling of imbalanced datasets, and an API-first backend architecture for real-time inference.

Machine Learning Pipeline (`notebooks/ML.ipynb`)

- **Data Leakage Prevention:** We enforce strict target exclusion. Metrics directly mathematically aligned to the label output bounds (like `radius_earth`, `eq_temp_k`, and `log_stellar_flux`) are removed prior to training to ensure proxy generalizability.
- **Class Imbalance Handling:** The dataset historically holds a 160:1 negative-to-positive ratio in planetary habitability. We employed **SMOTE (Synthetic Minority Over-sampling Technique)**. Critically, to preserve real-world validation integrity, SMOTE is scoped *strictly inside* the Cross-Validation folds using the `imblearn` pipeline, preventing synthetic data leakage into test sets.
- **Ensemble Modeling:** The apex model is a Soft-Voting Classifier bridging Random Forest, Logistic Regression, and XGBoost estimators. It maximizes the **AUPRC (Area Under Precision-Recall Curve)**, yielding the highest confidence rates for minority "Habitable" predictions.
- **Model Persistence:** Extracted models and scalers are pipelined together and binary-serialized using `joblib` into the `models/` directory for fast bootstrapping in production.

General Application Architecture

The overall application follows a decoupled three-tier architecture:

1. **Inference Backend (Python/Flask):** Listens for parameter payloads, unwraps the serialized ensemble classifiers, maps incoming JSON arrays to the expected `X_COLS` dataframe constraints, and runs probabilistic inference.
 2. **Interactive UI (React.js):** A client-side state manager driving an interactive dashboard. The UI groups metrics visually, offers quick planetary templates (Earth, Gas Giant, Hot Planet), and graphs prediction probabilities on completion via Axios API calls.
-

2. Frontend Code Updates

The frontend interface dynamically renders based on model dependencies.

Modifications (in `frontend/app.js`):

- **Feature Compatibility Patch:** The newest ML Multi-class classification model rigorously expects `orbital_period` as a feature constraint.
 - **Form State Updates:** The frontend React tracking hooks (`setFormData`) and the physical `<InputForm />` array mappings were hard-updated to present `Orbital Period (Days)` .
 - **Pre-loads Configuration:** Built-in generic pre-loader templates were mapped securely to pass `orbital_period` dynamically per selection (e.g., Earth: 365.25 Days).
-

3. Backend Code Updates

The backend is responsible for API data validation and model delivery.

Modifications (in `backend/app.py` & `backend/test_api.py`):

- **Validation Constraints:** Updated backend payload verification logic to strictly throw `400 Bad Request` exceptions if users or client interfaces omit the new `orbital_period` feature.
 - **Port Re-binding:** Addressed endpoint disconnections by globally normalizing the application context and the Test Suite clients to communicate cleanly over **Port 5001**.
 - **Model Loaders:** Programmed dynamic fallback catching where if the specific binary model goes offline, the backend serves Soft-Fail feedback seamlessly back to the UI interface without hard-crashing the Flask framework.
-

4. Deployment Files

The deployment stack abstracts the system for unified cloud virtualization.

The required system configuration packages are successfully aligned:

- `requirements.txt` : Completely synchronized with dependencies (e.g., `imblearn` , `xgboost` , `scikit-learn` , `flask` , `flask-cors` , `pandas`).
 - `deployment/ Environment Configurations`: Contains the scaling containerization logic required when deploying the application to cloud servers like AWS/GCP (Dockerfiles, Docker Compose).
-

5. Working Link of the Application/Interface

To launch the integrated Exoplanet Application:

1. Start the Backend API Server:

Ensure you are in the project root (`B13-ExoHabitAI`), then run:

```
python backend/app.py
```

The server will boot and begin listening for API requests on `http://localhost:5001`.

2. Open the Frontend Interface:

Simply open the HTML interface directly in your browser:

Local Path: `file:///C:/Users/konal/OneDrive/Documents/GitHub/B13-ExoHabitAI/frontend/index.html`

By ensuring the backend is running while opening the frontend link above, the dashboard will natively connect to the API prediction clusters to classify exoplanets in real-time.